Accessing GPUs: Cloud and Clusters

Zhirui Hu

Department of Statistics

Oregon State University

04/24/2026

Where to find GPUs to use?

- COS/COE computing clusters
- Google Colab
- Cloud computing:
  - AWS EC2, GCP compute engine, Azure ML
- Dstack for self-hosted AI environments and cloud

COS/COE computing clusters

COS computing clusters

- Novus HPC Cluster
  - Connect to VPN if off-campus (cisco)
  - ssh user@novus.dri.oregonstate.edu

```
Partition list with Allocated/Idle/Other/Total number of cores
 PARTITION      AVAIL CPUS(A/I/O/T)     NODES(A/I/O/T) NODELIST
 dri.q*         up    1372/156/0/1528   33/0/0/33      dri[001-016],sun[01-17]
 preempt.q      up    1374/346/0/1720   34/3/0/37      dri[001-016],hazboun01,schneider[01-02],spp01,sun[01-17]
 test.q         up    0/16/0/16         0/2/0/2        dri[01-02]
 gpu.q          up    0/32/0/32         0/1/0/1        gpu001
 preempt-gpu.q  up    144/592/80/816    5/2/1/8        forsythe01,gpu001,zuehlsdorff[01-06]
 forsythe.q     up    0/256/0/256       0/1/0/1        forsythe01
 hazboun.q      up    2/126/0/128       1/0/0/1        hazboun01
 lazzati.q      up    240/96/0/336      5/1/0/6        lazzati[01,04-08]
 nmr.q          up    0/16/0/16         0/2/0/2        dri[01-02]
 schneider.q    up    0/48/0/48         0/2/0/2        schneider[01-02]
 spp.q          up    0/16/0/16         0/1/0/1        spp01
 zuehlsdorff.q  up    144/304/80/528    5/0/1/6        zuehlsdorff[01-06]
 bosun.q        up    0/24/0/24         0/1/0/1        bosun01
 cos-arm.q      up    0/72/0/72         0/1/0/1        cos-gh01
 cos-priority.  up    0/72/0/72         0/1/0/1        cos-gh01
```

GPUs on Novus HPC Cluster

```
bosun01 CPUS: 24, Memory: 62GB, Partitions: bosun.q, GPUs: gpu:k80:4
cos-gh01 CPUS: 72, Memory: 439GB, Partitions: cos-arm.q,cos-priority.q, GPUs: gpu:gh200:1
forsythe01 CPUS: 256, Memory: 994GB, Partitions: preempt-gpu.q,forsythe.q, GPUs: gpu:a40:2
gpu001 CPUS: 32, Memory: 55GB, Partitions: gpu.q,preempt-gpu.q, GPUs: gpu:tesla:4
zuehlsdorff01 CPUS: 104, Memory: 367GB, Partitions: preempt-gpu.q,zuehlsdorff.q, GPUs: gpu:rtx2080:7
zuehlsdorff02 CPUS: 104, Memory: 367GB, Partitions: preempt-gpu.q,zuehlsdorff.q, GPUs: gpu:rtx2080:8
zuehlsdorff03 CPUS: 80, Memory: 183GB, Partitions: preempt-gpu.q,zuehlsdorff.q, GPUs: gpu:rtx3080:4
zuehlsdorff04 CPUS: 80, Memory: 183GB, Partitions: preempt-gpu.q,zuehlsdorff.q, GPUs: gpu:rtx3080:4
zuehlsdorff05 CPUS: 80, Memory: 183GB, Partitions: preempt-gpu.q,zuehlsdorff.q, GPUs: gpu:rtx3080:4
zuehlsdorff06 Cores: 80, Memory: 183GB, Partitions: preempt-gpu.q,zuehlsdorff.q, GPUs: gpu:rtx3080:4
```

ssh gpu001
nvidia-smi

COE computing clusters

- Connect to VPN if off-campus
- Access: ssh user@submit.hpc.engr.oregonstate.edu
- Data storage: hpc-share (medium term storage, no backup)
  - scp / rsync for transferring data
- Load module:
  - module load python/3.10
  - module load cuda/12.2
  - module list (check current loaded modules)

Partition	Description/Owner	Accounts	Features (e.g. OS / GPU / CPU)
share	Shared resources, open to all users	ALL	el8, el9 / **m60, a40** / sandybridge/ivybridge, haswell/broadwell, skylake, epyc, avx2, avx512
dgxh	DGX-H systems	ALL	el9 / **h100, h200, vram80g, vram140g** / sapphire, avx512
dgx2	DGX-2 systems	ALL	el9 / **v100** / cascade, avx512
ampere	Resources for research group	ALL	el9 / **a40** / epyc, avx2
preempt	Low priority queue with access to all resources, but subject to preemption by higher priority jobs	ALL	ALL

nodestat dgx2

https://it.engineering.oregonstate.edu/hpc/slurm-howto

GPU	Arch	FP32 (TFLOPS) ▾	VRAM	Mem BW (GB/s)	TDP (W)	Primary use
Tesla K40	Kepler	4.3	12 GB	288	235 W	Legacy HPC, research
M60	Maxwell	9.6	16 GB	320	300 W	VDI, remote graphics
RTX 2080	Turing	10.1	8 GB	448	215 W	Gaming, prosumer ML
V100	Volta	15.7	32 GB	900	300 W	Deep learning, HPC
RTX 3080	Ampere	29.8	10 GB	760	320 W	Gaming, prosumer ML
A40	Ampere	37.4	48 GB	696	300 W	Pro vis, AI training
H100	Hopper	67	80 GB	3350	700 W	LLM training, inference
H200	Hopper	67	141 GB	4800	700 W	LLM training, inference

- For AI/ML: H200 > H100 >> A40 > V100 by a wide margin in bandwidth and VRAM.
- VRAM: fast, specialized type of RAM that sits directly on the GPU and stores data needed for immediate processing

What model can we run for 48GB VRAM?

- Model inference:
  - **70B Models (Quantized):** Llama 3.3 70B or Qwen2.5 72B will fit.
  - **Smaller Models (High Precision/long Context):** Llama 3.1 8B can run at FP16 with massive context windows (e.g., 100k+ tokens).
- Model training/fine-tuning:
  - **QLoRA (Fine-tuning):** Llama 3.3 70B barely or Qwen 32B (small batch size).
  - **LoRA (Fine-tuning):** Llama 3.1 8B comfortably.
  - **Full fine-tuning/Training**: 1B model easily.

This slide is from AI, subject to validation on cluster

Submitting jobs

- Interactive job: srun -p dgx2 --gres=gpu:1 --time=1:00:00 --pty zsh
- Batch job: sbatch myBatchFile.sh
- View job: squeue -u {username}
- Cancel job: scancel {jobid}

https://it.engineering.oregonstate.edu/hpc/slurm-howto

myBatchFile.sh

```
#!/bin/bash
#SBATCH -J helloWorld                    # name of job
#SBATCH -p share                         # name of partition or queue
#SBATCH -o helloWorld.out                # name of output file for this submission script
#SBATCH -e helloWorld.err                # name of error file for this submission script
#SBATCH -t 24:30:00          # time limit on job: 24 hours, 30 minutes (default
             12 hours)
#SBATCH --gres=gpu:1          # number of GPUs to request per node (default 0)
#SBATCH --cpus-per-task=8       # number of cores/threads per task (default 1)
#SBATCH --mem=20G             # request 20 gigabytes memory per node



# load any software environment module required for app (e.g. python, R, gcc, cuda)
module load software/version

# run my job (e.g. R, python)
mySoftwareExecutable
```

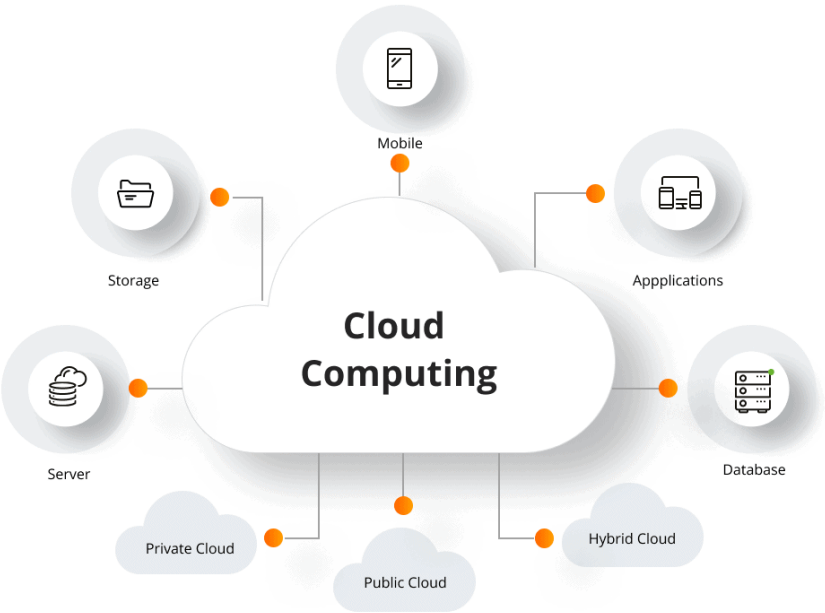
Cloud computing

Google colab

- Cloud-based Jupyter notebook
- Free access to GPUs and TPUs
- Free Pro account for students and educators
- **File system**: Temporary storage that clears when your session ends (you can mount Google Drive to persist files)
- **Libraries**: Most common data science libraries are pre-installed

Why Cloud Computing

- Run large statistical/ML workloads without managing physical hardware
- Virtual machine (VM)
  - a software-emulated computer running on shared hardware
  - root access to a "computer" you never physically touch



Pros & Cons of Cloud Computing

- Pros:
  - Scalability: e.g. run 10,000 simulations overnight, pay only for that time
  - Access GPUs, high-memory machines without owning them
  - Reproducibility: environment snapshots ensure running the exact same setup (but can change over time due to updates)
  - Easy to share data and codes
- Cons:
  - High (unexpected) cost: free tiers or trial credits
  - Data privacy and compliance
  - Steep learning curve

Basic steps to run jobs on cloud

- Launch a virtual server (called an *instance*)
  - Choose OS, CPU/GPU count, RAM, and storage.
- SSH to virtual server
- Install software and run jobs
- Download results or save to cloud storage
- Terminate instance
- Azure ML workspace creates a jupyter notebook, no need ssh

Instances

- AWS EC2
  - t3 — cheap, general purpose (dev work)
  - m6i — balanced CPU + memory
  - r6i — memory-optimised (large datasets)
  - p3/p4 — GPU instances for deep learning

  - On-demand — pay by the second, no commitment
  - Spot — up to 90% cheaper; instance may be interrupted
  - Reserved — 1–3 yr commitment, big discount

- GCP Compute Engine
  - E2 — cost-optimised, good for light jobs
  - N2 — general purpose, balanced
  - M3 — memory-optimised (up to 12 TB RAM)
  - A2/A3 — NVIDIA GPU machines
  - Preemptible VMs — equivalent of Spot, ~80% cheaper

Comparison of cloud service providers

Dimension	AWS EC2	GCP Compute Engine	Azure ML
Abstraction level	Raw VMs	Raw VMs	Managed ML platform
Best for	Flexible, general purpose compute	Sustained use discounts, Google-ecosystem	End-to-end ML pipelines
Softwares	Install anything manually; Jupyter through SSH tunnel	Install anything; Vertex AI Workbench for Jupyter	Managed Jupyter; conda envs built-in
Strengths	Largest instance catalogue, mature ecosystem	Fastest network infrastructure for data-heavy workloads, gcloud CLI is cleaner for reproduciblility	log metrics in dashboard, auto model selection (AutoML)

Data storage

Tiers	Name (AWS/GCP/Azure)
Store large files or datasets	S3 / GCS / Blob
Fast read/write during a job, Disk attached to instance	EBS / Persistent Disk / Azure Disk
Share data across instances	EFS / Filestore / Azure Files
Relational database	RDS / BigQuery / Azure SQL

What is Dstack?

Open-Source Orchestration

Dstack is an open-source platform that lets researchers request and run GPU workloads from any cloud provider or on-premise cluster — through a simple CLI and Python SDK.

Think of it as:

"Slurm for the cloud — but friendlier."

GPU-first design

Request A100s, H100s, or any GPU with one line

Container-based

Define your env with Docker or conda — reproducible

Multi-cloud & on-prem

AWS, GCP, Azure, Lambda, or your own cluster

Python & CLI friendly

Works with Jupyter, PyTorch, R, and more

https://dstack.ai/docs/guides/migration/slurm/#dstack

How Dstack Works — 3 Simple Steps

1

Define Your Task

Write a YAML config specifying your GPU type, Docker image, and script.

```
type: task
image: pytorch/pytorch:2.2
gpu: 1
commands:
  - python train.py
```

2

Submit with CLI

One command provisions the GPU and runs your job on any backend.

```
dstack run . \
  -f train.dstack.yml
# Streams logs in
# real-time
```

3

Get Results

Access outputs, attach VS Code or Jupyter, or pull artifacts locally.

```
dstack run . \
  --ide vscode
# Opens remote
# VS Code session
```